

Sijill — Build Spec

For Devin or any coding agent. Self-contained: build from this document alone. Where this spec and the PRD disagree, this spec wins for implementation; the PRD wins for positioning.

Objective

A proxy that sits in front of an OpenAI-compatible model endpoint, evaluates every call against a policy, and writes a signed, hash-chained record of it. An independent verifier proves the chain is intact or names the exact record that broke. A report generator produces a PDF for a date range. A dashboard shows records arriving live.

Everything runs locally on one machine. No cloud dependency at demo time.

Stack

Concern	Choice
Language	Python 3.11
Proxy	FastAPI with httpx async client. No blocking HTTP in request handlers
Model	Local runtime exposing an OpenAI-compatible API (e.g. Ollama at localhost:11434/v1) serving a small instruct model
Store	SQLite, file <code>sijill.db</code>
Hashing	SHA-256
Signing	Ed25519
Policy	YAML
Report	HTML template rendered to PDF
Dashboard	One static HTML page polling the API. No frontend framework

Repository layout

```

sijill/
  sijill/
    record.py          # schema, canonicalisation, hashing, signing — FROZEN at
H1
    store.py           # append-only SQLite access
    policy.py          # YAML load, rule evaluation
    proxy.py           # FastAPI app
    keys.py            # keypair generation and loading
  verifier/
    sijill_verify.py   # standalone CLI — must NOT import from sijill/
  report/
    generate.py
    template.html
  dashboard/
    index.html
  scripts/
    seed.py            # generate N calls through the proxy
    bench.py           # latency overhead measurement
    tamper.sh          # demo helper: edits one record via sqlite3
  policy.yaml
  tests/
  README.md

```

The verifier must not import from the `sijill/` package. It reimplements canonicalisation and verification from this spec. That independence is a product claim — the thing that checks the records is not the thing that wrote them.

Record schema

One table, `records`. One row per event.

Field	Type	Meaning
seq	integer, primary key	Starts at 1, increments by exactly 1
type	text	<code>inference</code> or <code>policy_change</code>
ts	text	ISO 8601 UTC, millisecond precision
model_id	text	Model name as requested

Field	Type	Meaning
model_digest	text	Content digest reported by the runtime
node_id	text	From policy file
node_region	text	Current region of the node
policy_id	text	From policy file
policy_hash	text	SHA-256 of the policy file bytes in force
approval_ref	text	From policy file
mode	text	<code>record</code> or <code>enforce</code> — the strictest mode among rules that fired
decision	text	<code>allow</code> , <code>flag</code> or <code>block</code>
rule_hits	text	JSON array of rule ids that fired
input_hash	text	SHA-256 of canonical request messages
output_hash	text	SHA-256 of response text; empty string if blocked
prev_hash	text	<code>record_hash</code> of seq-1; 64 zeros for seq 1
record_hash	text	See below
signature	text	Hex Ed25519 signature over <code>record_hash</code> bytes

For `policy_change` records, model and I/O fields are empty strings and `rule_hits` lists the rules whose mode changed.

Canonicalisation and hashing — exact

1. Take the record as a JSON object of every field **except** `record_hash` and `signature`.
2. Serialise with keys sorted, separators `,` and `:` with no whitespace, non-ASCII preserved, encoded UTF-8.
3. `record_hash` = lowercase hex SHA-256 of those bytes.
4. `signature` = Ed25519 signature of the 32 raw bytes of `record_hash` (decode the hex first), hex-encoded.

In Python: `json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=False).encode("utf-8")`. `seq` is serialised as an integer; every other field as a string.

Append-only rule

Application code only ever inserts. No UPDATE or DELETE statements anywhere in `sijill/`. The tamper demo edits the database directly with the sqlite3 CLI, from outside the application, which is the point.

Policy file

```
policy_id: gov-assistant
version_label: "2026-09-25.1"
approval_ref: "APR-0147"
node:
  id: "auh-node-01"
  region: "AE-AZ"
rules:
  - id: residency
    type: region_allowlist
    allow: ["AE-AZ", "AE-DU"]
    mode: enforce
  - id: approved_models
    type: model_allowlist
    allow_digests: ["<digest of the pulled model>"]
    mode: enforce
  - id: sensitive_terms
    type: keyword_flag
    terms: ["emirates id", "passport number"]
    mode: record
```

Rule semantics:

- `region_allowlist` fires when the node's current region is not in `allow`.
- `model_allowlist` fires when the model digest is not in `allow_digests`.
- `keyword_flag` fires when any term appears, case-insensitive, in the request messages.
- A rule firing in `record` mode: call proceeds, decision `flag`.

- A rule firing in `enforce` mode: call is not forwarded, proxy returns HTTP 403 with the record's `seq`, decision `block`.
- Nothing fires: decision `allow`, mode `record`.
- Every call is recorded regardless of decision. A block is evidence too.

Endpoints

Method and path	Purpose
POST <code>/v1/chat/completions</code>	OpenAI-compatible, non-streaming only. Evaluate, forward or block, seal, return
GET <code>/api/records?since=N</code>	Records with <code>seq</code> greater than <code>N</code> , for the dashboard
POST <code>/api/admin/policy/reload</code>	Re-read <code>policy.yaml</code> ; if any rule mode changed, write a <code>policy_change</code> record
POST <code>/api/admin/node/region</code>	Body <code>{"region": "..."} . Demo control for triggering residency</code>
GET <code>/api/report?from=&to=</code>	Returns the PDF for the range

Admin endpoints need no auth for the demo. State that openly; do not hide it.

Verifier CLI

```
python verifier/sijill_verify.py --db sijill.db --pubkey node.pub
```

Walks every record in `seq` order and checks, in this order:

1. `seq` is exactly previous + 1 → else `FAIL seq=N gap`
2. recomputed hash equals stored `record_hash` → else `FAIL seq=N content altered`
3. `prev_hash` equals previous record's `record_hash` → else `FAIL seq=N chain broken`
4. signature verifies against the public key → else `FAIL seq=N signature invalid`

Stops at the first failure. On success prints `PASS records=N first=<ts> last=<ts>`. Exit code 0 on pass, 1 on fail. Output must be large and readable on a projector; the demo shows it full-screen.

Acceptance tests — all must pass before H14

1. Seed 50 calls. Verifier passes.
2. Change `output_hash` of record 17 with `sqlite3`. Verifier fails at 17, `content altered`.
3. Change record 17's content and recompute its hash with the same algorithm, without the private key. Verifier fails at 17, `signature invalid`.
4. Delete record 17. Verifier fails at 18, `gap`.
5. Set node region to a disallowed value, make a call: HTTP 403, record written with decision `block`. Verifier still passes.
6. Change `sensitive_terms` mode to enforce and reload: a `policy_change` record is written. Verifier passes.
7. Latency: 100 calls, measure proxy overhead excluding model time. Report p50 and p95. Target p95 under 50 ms.

Test 3 is the most important. It shows an attacker who understands the hashing still cannot forge a record.

Report

One PDF, A4, for a date range: entity name, node, period, total calls by decision, policy hashes in force during the period with their approval references, verifier result run at generation time, and a table of every `flag` and `block`. Formal document styling — something an audit committee receives, not a developer printout.

Dashboard

One page. Polls every second. Shows the newest records as rows: seq, time, decision, rules hit, first 12 characters of `record_hash`. Blocks and flags visibly distinct. A counter of total records and the current node region. Nothing else.

Work split

Owner	Components	Why
Engineer	<code>record.py</code> , <code>store.py</code> , <code>keys.py</code> , <code>proxy.py</code> , <code>policy.py</code>	Correctness-critical. A subtle bug here makes the demo false

Owner	Components	Why
Devin	Verifier CLI and tests 1-6	Fully specified above; independent implementation is a feature
Devin	Report generator	Formatting work, low risk
Devin	Dashboard, <code>seed.py</code> , <code>bench.py</code> , <code>tamper.sh</code>	Parallelisable, easy to check
Devin	README with setup and demo steps	Last

Engineer commits `record.py` first, before anything else. Devin starts on the verifier from this spec in parallel and does not wait.

Session log

Keep `DEVIN_LOG.md` from the first task. For each task: what was delegated, what came back, what a human changed and why. It is shown to the judges. Reconstructing it afterwards does not work.

Out of scope — do not build

Authentication beyond what is stated. Multi-tenancy. Streaming responses. Policy authoring UI. Hardware attestation or TEE integration. Any database other than SQLite. Any feature not listed here — if an agent proposes one, the answer is no.

First prompt for Devin

Read the Build Spec in README.md. Build the verifier CLI in `verifier/sijill_verify.py` exactly as specified under "Canonicalisation and hashing" and "Verifier CLI". Do not import anything from the `sijill/` package. Write a fixture generator that creates a valid 50-record chain with a test keypair, then implement acceptance tests 1 through 4 against that fixture. Log your work in `DEVIN_LOG.md`. Stop and report when tests 1-4 pass.